Software Engineering

Unit 1 — Complete Exam-Ready Study Notes

Course: DU - 2304CS304
Based on: unit-1.pptx
Prepared for: Tanish Uchhadiya

Table of Contents

1. Introduction to Software
2. Characteristics of Software
3. Software Application Domains
4. Attributes of Good Software
5. Introduction to Software Engineering
6. Software Engineering as a Layered Technology
7. Software Myths
8. Software Process and Process Framework
9. Umbrella Activities
10. Generic Process Model and Task Set
11. Waterfall Model
12. Incremental Model
13. Evolutionary Models: Prototyping and Spiral
14. RAD Model
15. Agile Concept and Agile Manifesto
16. Scrum Model
17. Final Revision and Exam Practice

1. Introduction to Software

Definition: Software is a collection of instructions, data, and programs that tells a computer what to do and how to do it. It is the non-tangible (invisible) part of a computer system.

Hardware is the physical, touchable part of a computer, such as the keyboard, monitor, CPU, and memory. Software cannot be touched physically, but it controls the hardware and helps users complete tasks.

Software can be a mobile app, website, operating system, game, utility tool, or system program. Examples include WhatsApp, Windows, MS Word, Chrome, and banking apps.

Software as a Product and a Vehicle

- **As a product:** Software delivers computing power. It can create, manage, acquire, modify, display, or transmit information. For example, MS Word is a product.
- **As a vehicle:** Software is used to deliver another product or service. For example, MS Word helps create a document, so it delivers the document product.

Note: Microsoft Word is both a product (the software itself) and a vehicle (it creates documents).

Key Points

- Software = programs + data + instructions.
- Software controls hardware.
- Software is intangible; hardware is tangible.
- Software requires development, testing, and maintenance.

2. Characteristics of Software

Software has special characteristics that make it different from physical products.

2.1 Software Is Developed, Not Manufactured

Physical products such as cars or mobile phones are manufactured in factories. Software is not manufactured; it is developed through engineering activities such as analysis, design, coding, testing, and maintenance.

- **Analysis:** Understanding what the user needs.
- **Design:** Planning how the software will work.
- **Coding:** Writing the program.
- **Testing:** Checking whether the software works correctly.
- **Maintenance:** Fixing errors and adding improvements after release.

Example: A banking mobile app is designed by UI/UX experts, coded by developers, and tested by testers. It is engineered to perform tasks such as money transfer, balance checking, and bill payment.

2.2 Software Does Not Wear Out

Hardware wears out with time because physical parts become old or damaged. Software does not wear out physically, but it can deteriorate because of repeated changes.

When developers change software, new errors may be introduced. These errors increase the failure rate. After fixing them, the failure rate may return to normal, but another change can again increase failures. Slowly, the minimum failure rate rises; this is called software deterioration due to change.

Diagram explanation: The ideal curve remains almost flat after initial testing. The actual curve rises whenever a change is made, then falls after repair, but gradually moves upward over time.

3. Software Application Domains

Software is used in many areas. The main application domains are given below.

Domain	Meaning	Examples
System Software	Programs that service other programs and manage computer resources.	Operating systems, compilers, device drivers, file-management utilities, networking software.
Application Software	Stand-alone software that solves a specific business or user need.	Payroll system, banking software, hospital management system.
Engineering/Scientific Software	Software used for mathematical calculations and scientific work.	MATLAB, AutoCAD, astronomy software, weather software.
Embedded Software	Software inside a device that controls its features and functions.	Microwave controller, washing-machine controller, car ABS, dashboard display.
Product-line Software	Software made for many customers with a specific capability.	Inventory-management products, accounting packages.
Web/Mobile Applications	Network-centric browser-based or mobile applications.	Amazon, Flipkart, WhatsApp, Instagram, Gmail.
Artificial Intelligence Software	Software using non-numerical algorithms to solve complex problems.	ChatGPT, Google Gemini, facial recognition, self-driving cars, game-playing software.

Note: System software helps other software run; application software helps the user perform a task.

4. Attributes of Good Software

Good software should not only work; it should also be reliable, easy to use, secure, and easy to maintain.

Attribute	Meaning	Example
Correctness	Software meets all specified requirements and performs its intended functions accurately.	A payment app transfers the exact amount entered by the user.
Reliability	Software works consistently and has minimal unexpected failures.	The app works properly during normal daily use.
Efficiency	Software uses CPU, memory, and network effectively and gives fast response.	The app opens quickly without consuming excessive memory.
Usability	Software is easy to learn and use; the interface is simple and user-friendly.	A new user can order food without confusion.
Maintainability	Bugs can be fixed and features can be updated easily.	Well-documented code helps developers repair errors quickly.
Portability	Software can run on different hardware or operating systems with little modification.	An app works on Android, iOS, and web browsers.
Scalability	Software can handle more users without performance problems.	An e-commerce site works during a big sale with many users.
Security	Software protects data from unauthorized access and maintains privacy, integrity, and availability.	A banking app protects user passwords and transactions.
Reusability	Components can be used again in the same or different systems.	A login module is reused in several apps.
Interoperability	Software can work with other systems, applications, or services.	An app connects with a payment gateway using APIs.

5. Introduction to Software Engineering

Definition: Software engineering is the application of a systematic, well-organized, and measurable approach to the development, operation, and maintenance of software.

Software engineering uses a process and a collection of methods to build high-quality software. It helps teams build complex systems on time and with good quality. It also brings discipline to software development work.

The main work products of software engineering are programs, data, and supporting documents.

Why Software Engineering Is Needed

- Software projects are large and complex.
- Requirements may change during development.
- Teams need proper planning and coordination.
- Quality, security, and reliability are important.
- Maintenance is required after delivery.

6. Software Engineering as a Layered Technology

Software engineering is often shown as a layered approach. Each layer supports the layer above it. The foundation is quality focus.

Layer Diagram Explanation

Bottom layer: **Quality Focus** → above it: **Process** → above it: **Methods** → top layer: **Tools**.

Layer	Meaning	Examples
Quality Focus	The base of software engineering. Every activity should improve software quality.	A banking app must be secure and should not fail during transactions.
Process	A structured way to manage tasks, progress, quality, and changes.	Agile or Scrum process using sprints.
Methods	Techniques that guide how software is built.	Planning, designing, coding, testing, UML diagrams, unit testing, design patterns.
Tools	Software tools that automate or support development tasks.	Git, JIRA, Visual Studio Code, testing tools.

Note: When tools work together and share data, they form CASE: Computer-Aided Software Engineering.

Methods such as Total Quality Management (TQM) and Six Sigma help create a culture of continuous improvement.

7. Software Myths

A myth is something people believe to be true, but it is actually wrong or misleading. Software myths are divided into management myths, customer myths, and developer myths.

Management Myths

Myth	Reality
A book of standards is enough to build software.	Standards are useful only if people know them, use them, and keep them complete, practical, and up to date.
Adding more people to a late project will speed it up.	Adding people may delay the project because of training and coordination time. People should be added in a planned manner.
If a project is outsourced, management does not need to worry.	Outsourcing still requires proper management. Without management knowledge, problems can occur.

Customer Myths

Myth: Changing requirements later will not cost much.
Reality: Late changes are costly and time-consuming, especially after the system is built.

Myth: Software is easy to change after development.
Reality: Changes require time, testing, and may affect other parts of the system.

Developer Myths

Myth	Reality
Software quality cannot be judged until it runs.	Quality can be checked early through technical reviews, which often find more issues than testing alone.
Only working software matters.	Documents, models, and plans are also essential for success and future support.
Software engineering means unnecessary paperwork and slows development.	Software engineering focuses on quality. Better quality reduces rework and can speed up delivery.
Once the program works, the job is done.	Most work continues after delivery: bug fixing, updates, and support.

8. Software Process and Process Framework

Process: A process is a collection of activities, actions, and tasks performed to create a work product. SDLC is an example of a software process.

Term	Meaning	Example
Activity	A broad objective performed in a project.	Design phase: design system architecture and UI.
Action	A set of tasks that produces a major work product.	Create UI wireframes: sketch screens, define navigation, choose UI components.
Task	A small, well-defined activity with a clear outcome.	Design the login screen.

Process Framework

A process framework establishes the foundation for a complete software engineering process. It includes key activities that apply to all software projects, regardless of size or complexity.

The five generic framework activities are:

- **Communication:** Gather requirements from clients, stakeholders, and users. Example: understand login, restaurant listing, ordering, payment, and tracking features for a food-delivery app.
- **Planning:** Prepare schedule, allocate resources, and estimate cost. Example: create a timeline for UI design, backend development, and testing.
- **Modeling:** Design system architecture, UI wireframes, and data structures. Example: design screens and database tables for users, orders, and restaurants.
- **Construction:** Write code and test the software. Example: build the UI, develop APIs, and perform unit and integration testing.
- **Deployment:** Deliver the software to users and maintain it. Example: publish an app on Google Play Store and release updates based on feedback.

Note: These activities are repeated in cycles. Each cycle delivers part of the software, and every new version makes the software more complete.

9. Umbrella Activities

Umbrella activities are supporting activities that continue throughout the software project. They help manage progress, ensure quality, handle changes, and reduce risks.

Umbrella Activity	Purpose	Example
Software Project Tracking and Control	Keeps the project on schedule by checking progress and adjusting plans.	Using Jira or Trello to track tasks in a mobile-app project.
Risk Management	Identifies and handles possible problems before they affect the project.	Planning a backup API if the main restaurant-data API fails.
Software Quality Assurance (SQA)	Ensures the software meets quality standards through checks and testing.	Regular unit and UI testing to ensure orders work correctly.
Technical Reviews	Reviews code, design, and documents to find errors early.	Reviewing UI designs before coding; GitHub pull-request code reviews.
Measurement	Collects data to improve development and quality.	Tracking crashes, response time, bug count, and code coverage.
Software Configuration Management (SCM)	Manages changes in code and documents without confusion.	Using GitHub to manage updates and roll back faulty changes.
Reusability Management	Uses existing components to save time and effort.	Reusing login screens or payment modules in multiple apps.
Work Product Preparation and Production	Creates and organizes project outputs.	Preparing wireframes, test cases, user manuals, and deployment guides.

10. Generic Process Model and Task Set

In a generic process model, every framework activity contains software engineering actions. Every action is defined by a task set.

A **task set** is the list of steps required to complete a software engineering action. It identifies work tasks, work products, quality-assurance points, and project milestones.

Process flow shows the order and timing of activities, actions, and tasks. It helps the team decide what to do first, what to do next, and how long each step may take.

Example: Task Set for Requirements Gathering

- List the stakeholders involved in the project.
- Call a meeting with them.
- Ask everyone to share required features.
- Discuss and prepare one final requirements list.
- Decide which features are most important.
- Identify unclear points.

11. Waterfall Model

The Waterfall Model is also called the **classic life cycle** or **linear sequential model**. In this model, software development follows a straight and clear path from communication to deployment.

Flow: Communication → Planning → Modeling → Construction → Deployment. Each phase must be completed before the next phase begins.

When to Use

- Requirements are well understood and stable.
- Only small updates or changes are being made to an existing system.
- A new project has clearly defined needs.

Why It May Fail

- Real projects rarely follow a strict sequence.
- Customers often cannot define all requirements at the beginning.
- Customers must wait a long time to see the working product.
- If a major mistake is found late, fixing it becomes very costly.
- Team members must wait for dependent tasks to be completed.

Quick Comparison

Aspect	Waterfall Model
Approach	Linear and sequential
Requirement change	Difficult after a phase is completed
Customer feedback	Mostly received at the end
Best use	Stable and clearly defined requirements
Risk handling	Weak for uncertain or complex projects

12. Incremental Model

The incremental model delivers the software in several releases called **increments**. Each increment provides progressively more functionality to the customer.

This model is useful when a limited set of features must be delivered quickly at first, and more features can be added later.

Example: Word-Processing Software

- First increment: basic file management, editing, and document creation.
- Second increment: advanced editing and document-production features.
- Third increment: spelling and grammar checking.
- Fourth increment: advanced page-layout capability.

Waterfall vs Incremental

Aspect	Waterfall Model	Incremental Model
Delivery	Complete product at the end	Part of the product in each increment
Customer feedback	Late	Early and after every increment
Flexibility	Low	Higher
Best use	Stable requirements	Early delivery with future expansion

13. Evolutionary Process Models

Evolutionary process models produce an increasingly complete version of software with each iteration. They are useful when business needs and requirements change during development.

These models allow a basic version to be released quickly, then improve it in multiple versions. The two common evolutionary models are the Prototyping Model and the Spiral Model.

13.1 Prototyping Model

A prototype is a basic working model of the software. It focuses mainly on user-visible parts such as screens and outputs. It is used when requirements are unclear or users know what they want but cannot explain detailed features.

Working: Understand basic requirements → develop a quick prototype → user gives feedback → refine requirements → repeat until requirements are clear → develop the final system.

Purpose: Clarify requirements, explore design options, and reduce misunderstandings before full development.

Problems with Prototyping

- **False impression of completeness:** Users may think the prototype is the final product, even though it may lack quality and structure.
- **Pressure to release unfinished software:** Stakeholders may demand quick launch instead of a proper rebuild.
- **Technical compromises:** Developers may use quick solutions that become difficult to replace later.

13.2 Spiral Model

The Spiral Model combines the iterative nature of prototyping with the structured approach of the waterfall model. It is suitable for complex and high-risk projects because it focuses on identifying and reducing risk in every phase.

Software is developed in cycles called spirals. Each loop includes planning, risk analysis, engineering, testing, and evaluation. Early cycles may produce prototypes, while later cycles produce more complete versions.

Anchor points are checkpoints, such as requirement review or design approval, used to ensure stakeholder agreement and project feasibility.

Prototyping vs Spiral

Aspect	Prototyping Model	Spiral Model
Main purpose	Clarify unclear requirements	Manage high-risk complex projects
Approach	Build a quick basic model and improve it	Repeat risk-driven cycles
Risk analysis	Not a major focus	Central focus in every cycle
Best use	Unclear user requirements	Large, costly, or risky projects

14. RAD Model

RAD (Rapid Application Development) emphasizes quick development and delivery of high-quality software using component-based construction.

RAD depends heavily on prototypes and user feedback to refine requirements and functionality quickly. It uses reusable components, tools, and templates to reduce development time and effort.

RAD Phases

- **Requirements Planning:** Understand the requirements and plan the project.
- **User Design:** Users interact with prototypes and give feedback.
- **Construction:** Developers build the system using reusable components and tools.
- **Deployment:** The final product is delivered to users.

When RAD Works Best

- The project is not too complex.
- Requirements are clear and well understood.
- The team is highly skilled.
- There is strong communication between developers and users.

15. Agile Concept and Agile Manifesto

Agile software engineering combines a philosophy and development guidelines. It encourages customer satisfaction and early incremental delivery of software.

Agile uses small, highly motivated teams, informal methods, minimal work products, and overall development simplicity. It focuses on delivery and continuous communication between developers and customers.

Cost of Change

Changes during requirement gathering are low-cost and easy to manage. Changes in later stages are more expensive because they may require redesign, recoding, and retesting. Therefore, the cost of change increases as the project progresses.

Agile Manifesto

The Agile Manifesto was written by 17 software-development practitioners in February 2001. It gives four core values and 12 principles for Agile software development.

Value	Meaning
Individuals and interactions over processes and tools	People and communication are more important than rigid procedures or tools.
Working software over comprehensive documentation	Deliver useful working software; keep documentation light and helpful.
Customer collaboration over contract negotiation	Work closely with customers because requirements and priorities can change.
Responding to change over following a plan	Accept useful changes instead of strictly following a fixed plan.

Note: The Agile Manifesto does not say documentation, processes, contracts, or plans are useless. It says the items on the left are valued more.

Agile Principle Themes

Theme	Focus
Value Delivery	Deliver valuable software early and frequently; working software is the main measure of progress.
Business Collaboration	Customers, business people, and developers work together daily and welcome changing requirements.
Team Dynamics and Culture	Build motivated, trusted, self-organizing teams; use face-to-face communication and maintain a constant pace.
Retrospectives and Continuous Learning	The team regularly reflects on what is working and improves its process.

16. Scrum Model

Scrum is an Agile software-development method. Its principles match the Agile Manifesto and guide development activities within a process.

Scrum Term	Meaning
Product Backlog	A prioritized list of project requirements or features that provide customer value. Items can be added at any time, and the product manager updates priorities.
Sprint	A fixed timebox, usually 7 to 30 days and mostly 2 weeks, in which the team completes selected backlog work. Changes are not introduced during a sprint.
Scrum Meeting	A short daily meeting, usually 15 minutes, where team members discuss progress, difficulties, and plans.
Scrum Master	The team leader who leads Scrum meetings and helps the team identify and remove problems.
Sprint Goal	The requirement or objective the team aims to achieve during the sprint.

Three Daily Scrum Questions

- What did you do since the last team meeting?
- What difficulties are you facing?
- What do you plan to complete by the next team meeting?

Note: Scrum provides a short-term but stable working environment because sprint requirements do not change during the sprint.

17. Final Revision and Exam Practice

Important Definitions

- **Software:** A collection of instructions, data, and programs used to operate a computer and perform tasks.
- **Software Engineering:** A systematic, well-organized, and measurable approach to developing, operating, and maintaining software.
- **Process:** A collection of activities, actions, and tasks used to create a work product.
- **Process Framework:** The foundation for a complete software engineering process, containing communication, planning, modeling, construction, and deployment.
- **Umbrella Activities:** Supporting activities that run throughout the project, such as risk management, SQA, measurement, and SCM.
- **Process Model:** A roadmap that defines the flow of activities, actions, tasks, iteration, and work products.
- **Sprint:** A fixed time period, usually 7 to 30 days, in which a Scrum team completes selected work.
- **Backlog:** A prioritized list of requirements or features that provide customer value.

Model Comparison

Model	Best When	Main Limitation
Waterfall	Requirements are stable and clear	Poor flexibility; late customer feedback
Incremental	Early partial delivery is needed	Requires good planning of increments
Prototyping	Requirements are unclear	Users may think prototype is final product
Spiral	Project is complex and risky	Costly and requires risk-analysis expertise
RAD	Requirements are clear and team is skilled	Not suitable for very complex projects
Agile/Scrum	Requirements change frequently	Needs active customer involvement

Short-Answer Questions

- **Q1. Define software.**
  Answer: Software is a collection of instructions, data, and programs that tells hardware what to do and how to do it.
- **Q2. What is software engineering?**
  Answer: Software engineering is the application of a systematic, well-organized, and measurable approach to the development, operation, and maintenance of software.
- **Q3. What is a process framework?**
  Answer: A process framework is the foundation of the software engineering process. It includes communication, planning, modeling, construction, and deployment.
- **Q4. What are umbrella activities?**
  Answer: Umbrella activities are supporting activities performed throughout the project, such as risk management, SQA, technical reviews, measurement, SCM, and reusability management.

- **Q5. What is a sprint in Scrum?**
  Answer: A sprint is a fixed development timebox, usually 7 to 30 days, in which the Scrum team completes selected backlog items.

Long-Answer Questions

- Explain software engineering as a layered technology with a diagram.
- Explain the five generic process-framework activities with examples.
- Explain any five umbrella activities with suitable examples.
- Explain the Waterfall Model, its advantages, limitations, and suitable applications.
- Explain the Incremental Model with an example.
- Compare the Prototyping Model and Spiral Model.
- Explain RAD Model and state when it is suitable.
- Explain the four values of the Agile Manifesto.
- Explain Scrum terms: product backlog, sprint, Scrum meeting, and Scrum master.

MCQs

Software is a collection of:

A) Only hardware
B) Instructions, data, and programs
C) Physical devices
D) Only users

Correct answer: B. Software includes programs, data, and instructions.

Which is a system software example?

A) MS Word
B) WhatsApp
C) Windows operating system
D) Payroll system

Correct answer: C. An operating system is system software.

Which attribute means software can run on different platforms?

A) Usability
B) Portability
C) Security
D) Efficiency

Correct answer: B. Portability means running on different hardware or operating systems.

Which model is best for unclear requirements?

A) Waterfall
B) Prototyping
C) RAD
D) Big Bang

Correct answer: B. Prototyping helps clarify unclear requirements through user feedback.

Which model focuses strongly on risk analysis?

A) Waterfall
B) Incremental
C) Spiral
D) RAD

Correct answer: C. The Spiral Model performs risk analysis in every cycle.

In Scrum, a sprint is usually:

A) 1 day
B) 7 to 30 days
C) 6 months
D) Unlimited

Correct answer: B. A sprint is a fixed timebox of 7 to 30 days, mostly 2 weeks.

Which is NOT a generic framework activity?

A) Communication
B) Planning
C) Modeling
D) Marketing

Correct answer: D. The five framework activities are communication, planning, modeling, construction, and deployment.

Which Agile value gives more importance to customer cooperation?

A) Working software
B) Customer collaboration
C) Processes and tools
D) Comprehensive documentation

Correct answer: B. Customer collaboration is preferred over contract negotiation.

Last-Minute Checklist

- Know the definition and examples of software.
- Remember that software is developed, not manufactured.
- Understand why software deteriorates due to changes.
- Learn all seven software-application domains with examples.
- Learn all attributes of good software.

- Remember the four layers: quality focus, process, methods, and tools.
- Learn management, customer, and developer myths with realities.
- Memorize the five framework activities.
- Learn all umbrella activities.
- Compare Waterfall, Incremental, Prototyping, Spiral, RAD, Agile, and Scrum.
- Remember the four Agile Manifesto values and Scrum terms.